If we evaluated the performance of the neural network using these random weights, we would expect it to be the same as random guessing. The learning algorithm improves the performance of the network by gradually changing each weight in the proper direction. This is called an **iterative** procedure, and is controlled in the program by the FOR-NEXT loop in lines 270-400. Each iteration makes the weights slightly more efficient at separating the target from the nontarget examples. The iteration loop is usually carried out until no further improvement is being made. In typical neural networks, this may be anywhere from ten to ten-thousand iterations, but a few hundred is common. This example carries out 800 iterations.

In order for this iterative strategy to work, there must be a *single* parameter that describes how well the system is currently performing. The variable ESUM (for error sum) serves this function in the program. The first action inside the iteration loop is to set ESUM to zero (line 290) so that it can be used as an accumulator. At the end of each iteration, the value of ESUM is printed to the video screen (line 380), so that the operator can insure that progress is being made. The value of ESUM will start high, and gradually decrease as the neural network is trained to recognize the targets. Figure 26-9 shows examples of how ESUM decreases as the iterations proceed.

All 260 images in the training set are evaluated during each iteration, as controlled by the FOR-NEXT loop in lines 310-360. Subroutine 1000 is used to retrieve images from the database of examples. Since this is not something of particular interest here, we will only describe the parameters passed to and from this subroutine. Subroutine 1000 is entered with the parameter, LETTER%, being between 1 and 260. Upon return, the input node values, X1[1] to X1[100], contain the pixel values for the image in the database corresponding to LETTER%. The bias node value, X1[101], is always returned with a constant value of *one*. Subroutine 1000 also returns another parameter, CORRECT. This contains the desired output value of the network for this particular letter. That is, if the letter in the image is a vowel, CORRECT will be returned with a value of *one*. If the letter in the image is not a vowel, CORRECT will be returned with a value of *zero*.

After the image being worked on is loaded into X1[1] through X1[100], subroutine 2000 passes the data through the current neural network to produce the output node value, X3. In other words, subroutine 2000 is the same as the program in Table 26-1, except for a different number of nodes in each layer. This subroutine also calculates how well the current network identifies the letter as a target or a nontarget. In line 2210, the variable ELET (for error-letter) is calculated as the difference between the output value actually generated, X3, and the desired value, CORRECT. This makes ELET a value between -1 and 1. All of the 260 values for ELET are combined (line 340) to form ESUM, the total squared error of the network for the entire training set.

Line 2220 shows an option that is often included when calculating the error: assigning a different *importance* to the errors for targets and nontargets. For example, recall the cancer example presented earlier in this chapter,